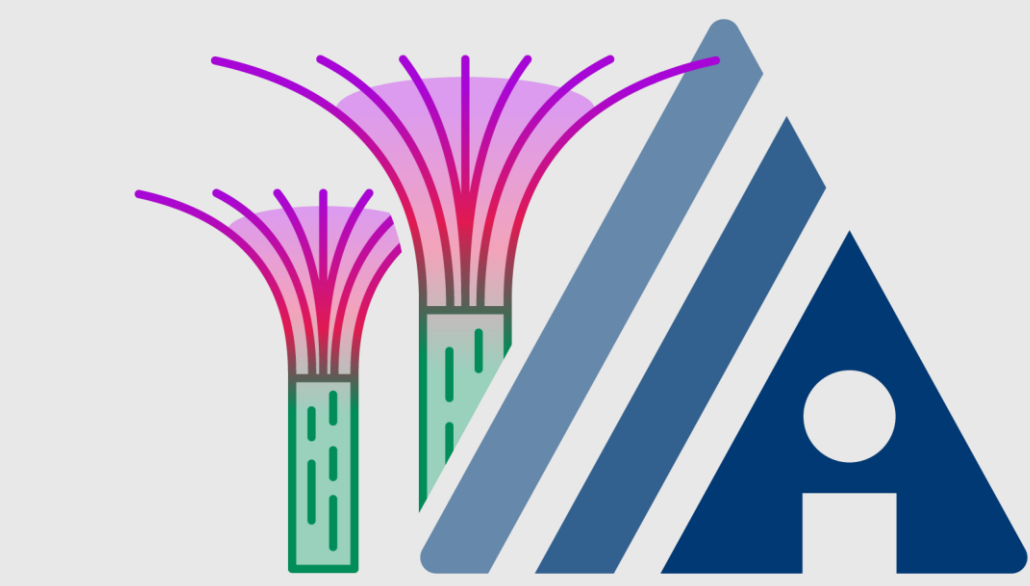




AAAI-26 / IAAI-26 / EAAI-26

JANUARY 20-27, 2026 | SINGAPORE

ORAL PRESENTATION

MOTIF: Multi-strategy Optimization via Turn-based Interactive Framework

Nguyen Viet Tuan Kiet¹, Tung Dao¹, Cong Dao Tran², Huynh Thi Thanh Binh^{1*}

¹Hanoi University of Science and Technology, Vietnam

²FPT Software AI Center, Vietnam

*Corresponding author: binhht@soict.hust.edu.vn



SOICT



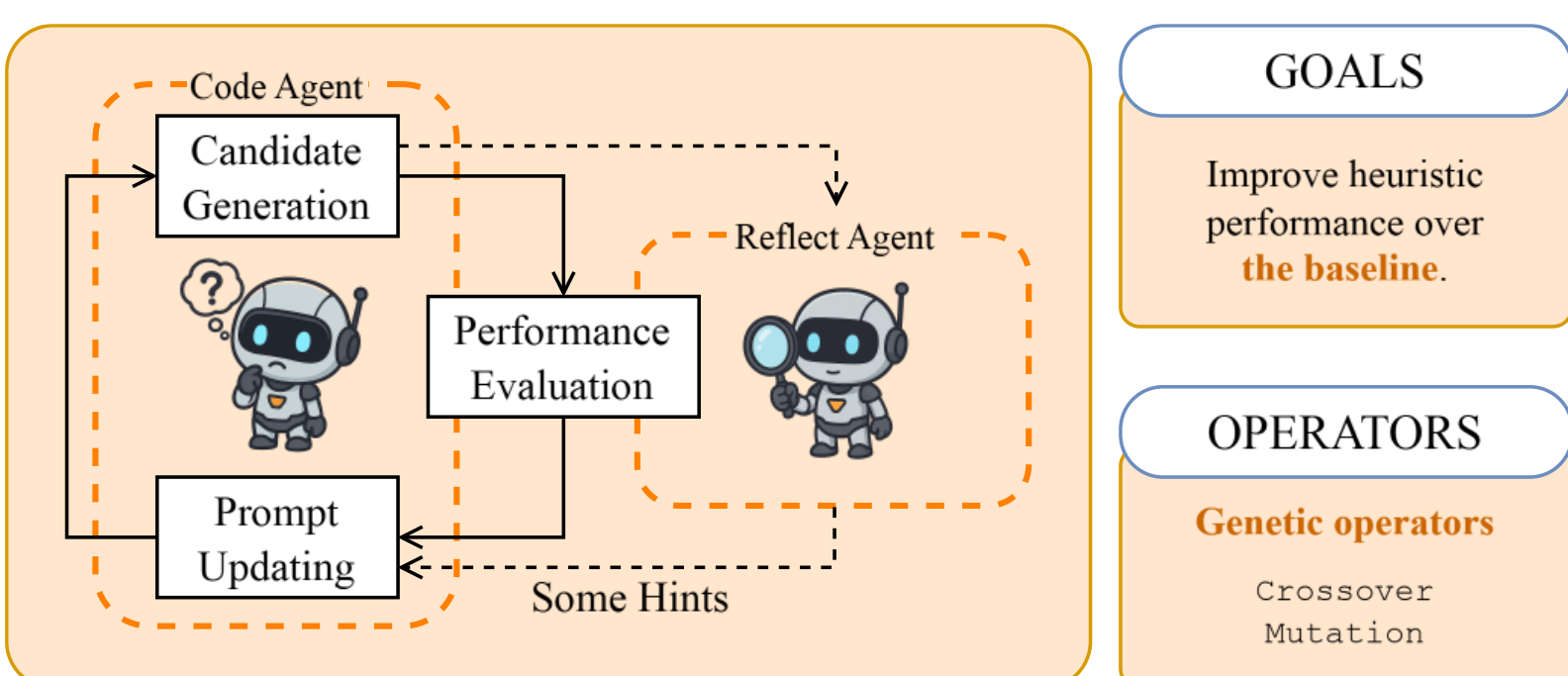
OUR CODEBASE

INTRODUCTION

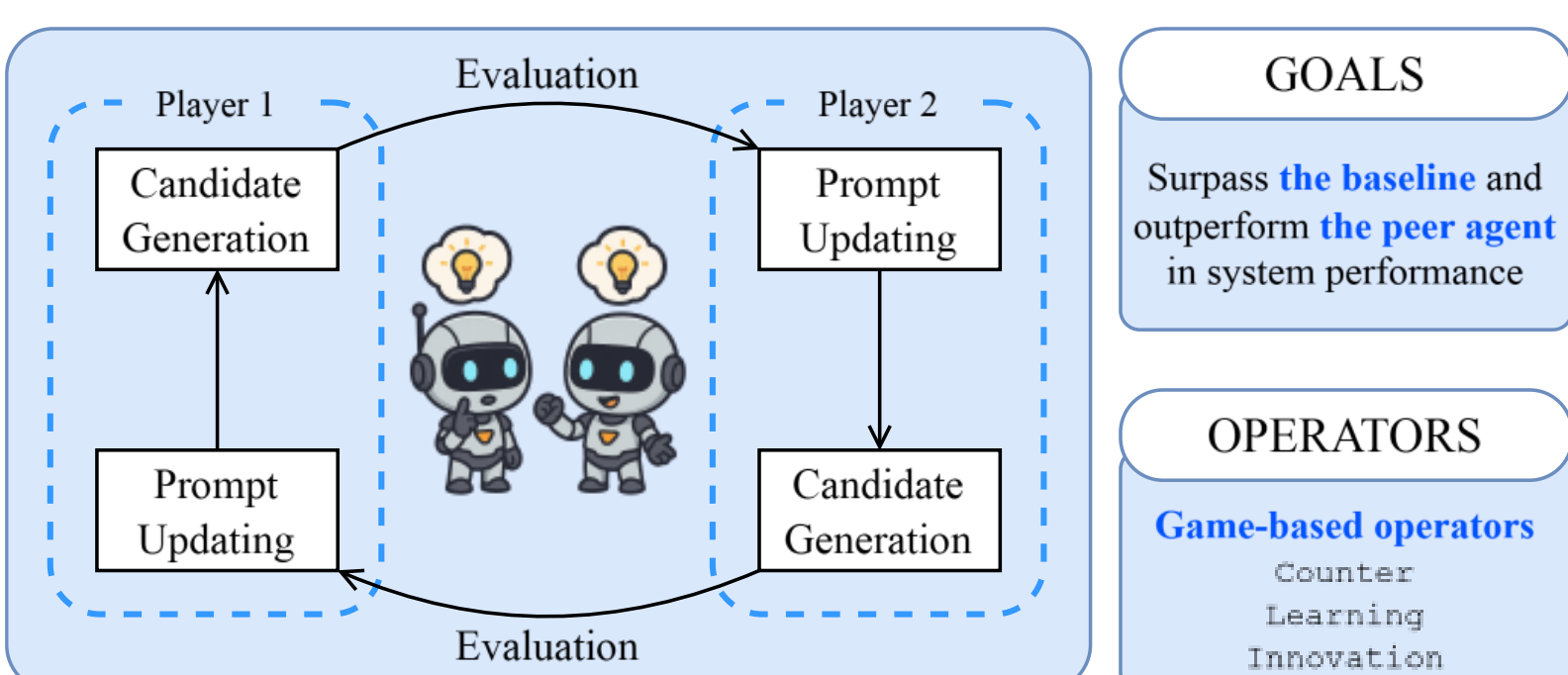
Approximate solvers for NP-hard Combinatorial Optimization Problems (COPs) are traditionally built from multiple hand-crafted components. Recent progress in Automatic Heuristic Design (AHD) uses LLMs to automate the design of *heuristic scoring functions*, yielding notable gains - but still assumes that improving a single component is sufficient. This narrow focus constrains exploration and limits creativity.

MOTIF breaks from this paradigm!

We introduce the *first framework* that treats solver design as *multi-strategy optimization*, enabling the joint search over *multiple* interacting components. This broader perspective expands the design space and uncovers solver improvements unreachable by single-strategy methods.



(a) Monolithic reflective pipeline



(b) Turn-based interactive framework (ours)

Prior Works [1, 2, 3, 4]

- **Population-based search** prune ideas early.
- **Crossover/mutation** limit diversity and rely on initial seeds.
- **One-way reflection** gives only single-sided feedback.
- **Single-heuristic focus** leaves most solver components untouched.

Our Work

- **Multi-strategy optimization** across multiple solver components.
- **Two-agent, turn-based interaction.**
- **New directional operators** that leverage competition and collaboration.
- **Enables broader search and stronger component synergy** than old methods.

MULTI-STRATEGY OPTIMIZATION

domain, instance, solution

A COP domain d consists of:

- a space of instances $\mathcal{X}_d$
- a global solution space $\mathcal{Y}_d$
- an objective function $f_d: \mathcal{X}_d \times \mathcal{Y}_d \rightarrow \mathbb{R}$

For each instance $\mathbf{x}$, the optimal solution is

$$\mathbf{y}^* = \arg \min_{\mathbf{y} \in \mathcal{Y}_d} f_d(\mathbf{x}, \mathbf{y})$$

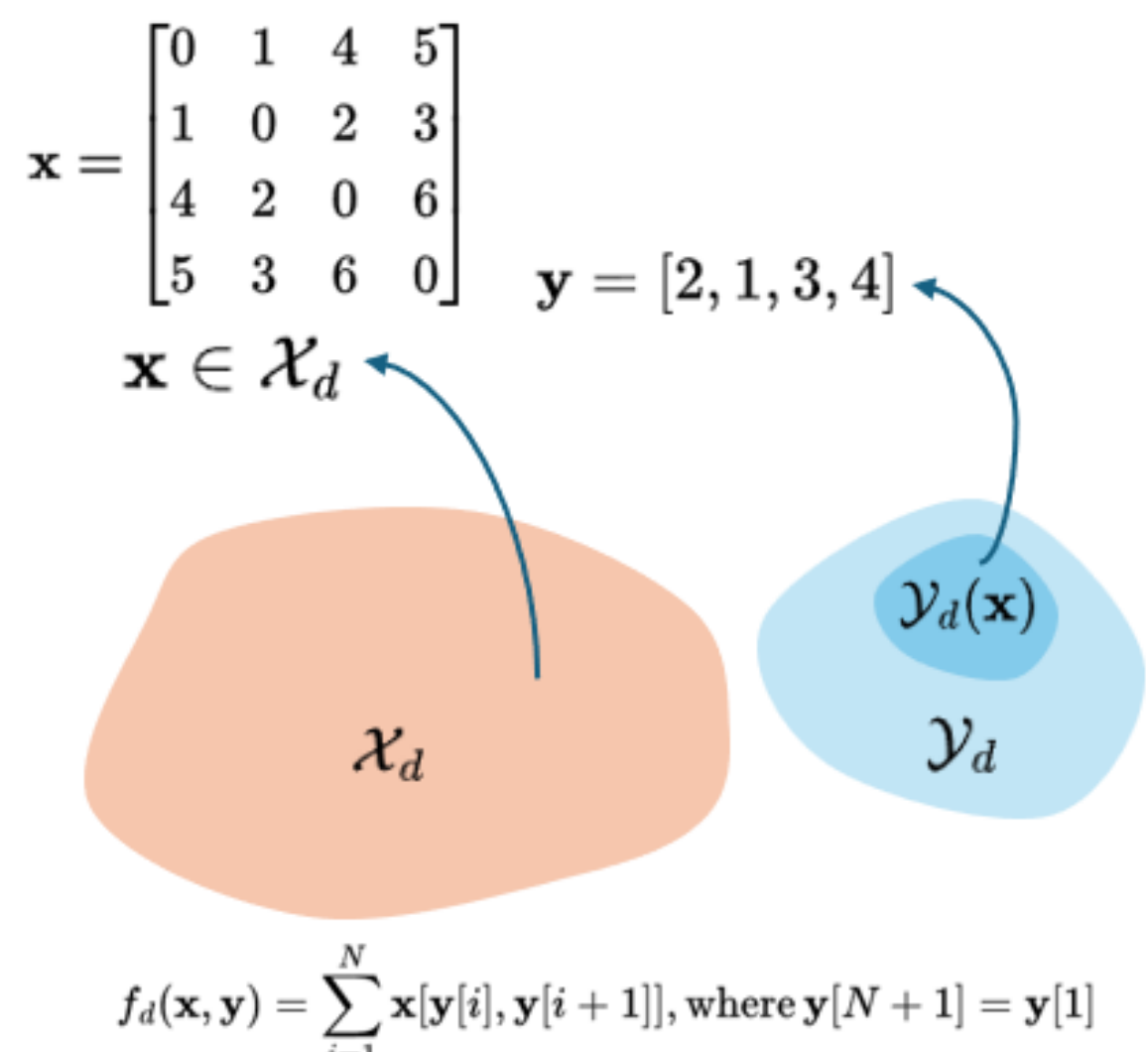


Figure 1. In the TSP domain, each instance is a distance matrix between cities, and each solution is a permutation of the cities.

Figure 3. An illustration of 2 versions of the transition-matrix computation strategy in ACO. They share *the same function signature and purpose* but differ in their internal logic and hyperparameters.

Multi-strategy optimization seeks the best joint configuration of all components:

$$(\pi_1^*, \pi_2^*, \dots, \pi_K^*) = \arg \min_{\pi_1, \pi_2, \dots, \pi_K} \mathbb{E}_{\mathbf{x} \sim \mathcal{X}_d} \left[f_d(\mathbf{x}, s(\mathbf{x} | \pi_1, \pi_2, \dots, \pi_K)) \right]$$

solver's performance

strategy, solver

A solver s is built from K strategies $\pi_1, \dots, \pi_K$

Each strategy π :

- acts as a supporting routine inside the solver
- belongs to a strategy space $\mathcal{S}_\pi$, containing all functions with the same signature

Together, these strategies produce the solution

$$\mathbf{y} = s(\mathbf{x} | \pi_1, \pi_2, \dots, \pi_K)$$

Algorithm 2: General ACO Procedure for COPs

```

1: Input: Problem-specific data  $\mathcal{I}$ , #ants  $M$ , #iterations  $T$ 
2: Strategy 1:  $H, P \leftarrow \text{INITIALIZE}(\mathcal{I})$  ( $H$ : heuristic,  $P$ : pheromone)
3: Initialize BestSolution
4: for  $t = 1$  to  $T$  do
5:   Strategy 2:  $W \leftarrow \text{COMPUTEPROBABILITIES}(H, P, t, T)$ 
6:   Solutions  $\leftarrow \text{CONSTRUCTSOLUTIONS}(W, \mathcal{I}, M)$ 
7:   Costs  $\leftarrow \text{EVALUATE}(\text{Solutions}, \mathcal{I})$ 
8:   Strategy 3:  $P \leftarrow \text{UPDATEPHEROMONE}(P, \text{Solutions}, \text{Costs}, t, T)$ 
9:   Update BestSolution if improved
10: end for
11: Output: BestSolution

```

Figure 2. The general ACO algorithmic framework for COPs is supported by 3 strategies, which are components containing expert-designed formulas.

```

def compute(heuristics, pheromones, t, T):
    alpha = 1.0
    beta = 2.0
    probabilities = pow(pheromones, alpha) * pow(heuristics, beta)
    return probabilities

```

```

def compute(heuristics, pheromones, t, T):
    alpha = t / T
    beta = 2.0 - alpha
    probabilities = pheromones / (heuristics ** beta)
    return probabilities

```

PROPOSED METHOD

Figure 5. The two-agent game in MOTIF unfolds in 2 rounds, progressing from **local** to **global** objectives as the context expands from **individual components** to the **full solver system**.

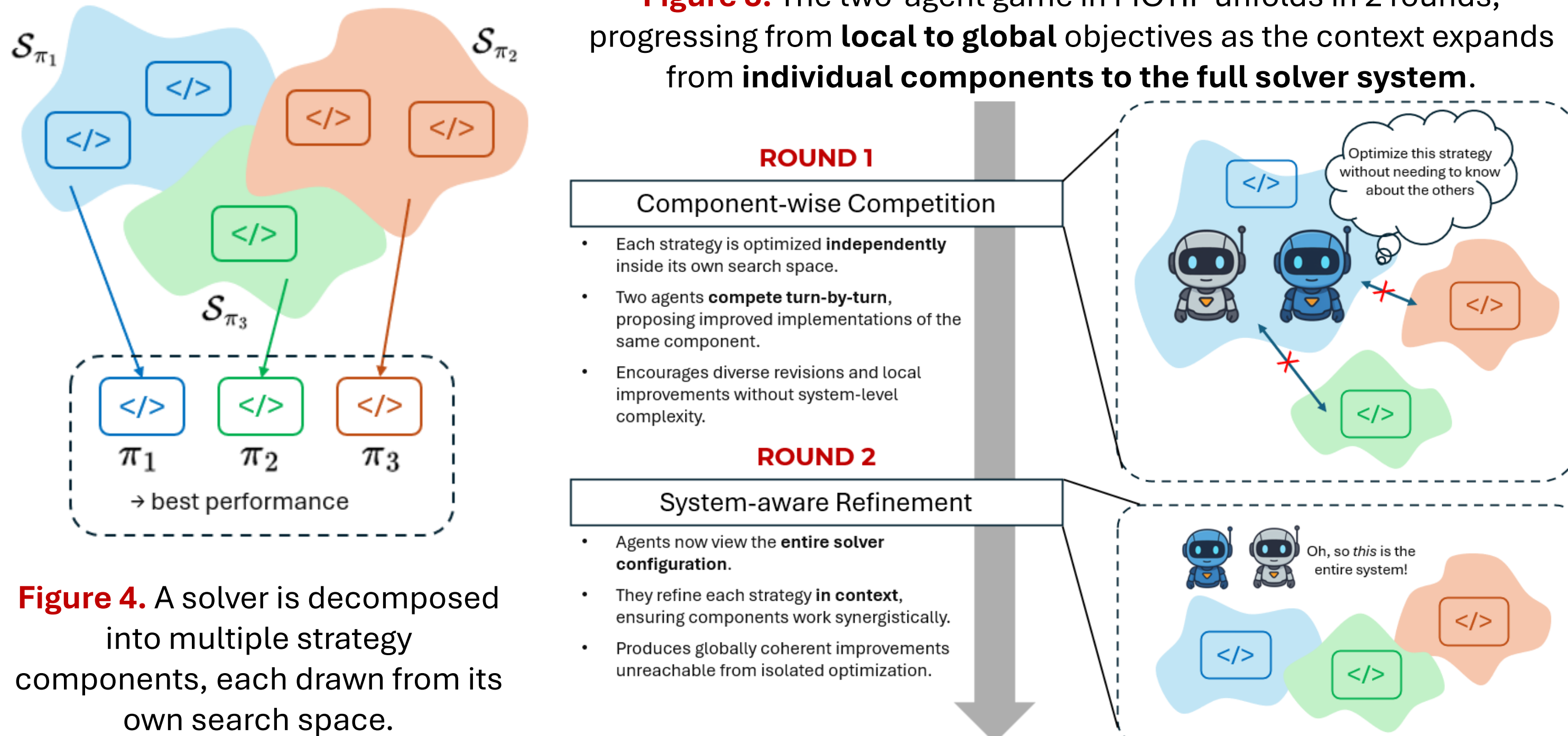


Figure 4. A solver is decomposed into multiple strategy components, each drawn from its own search space.

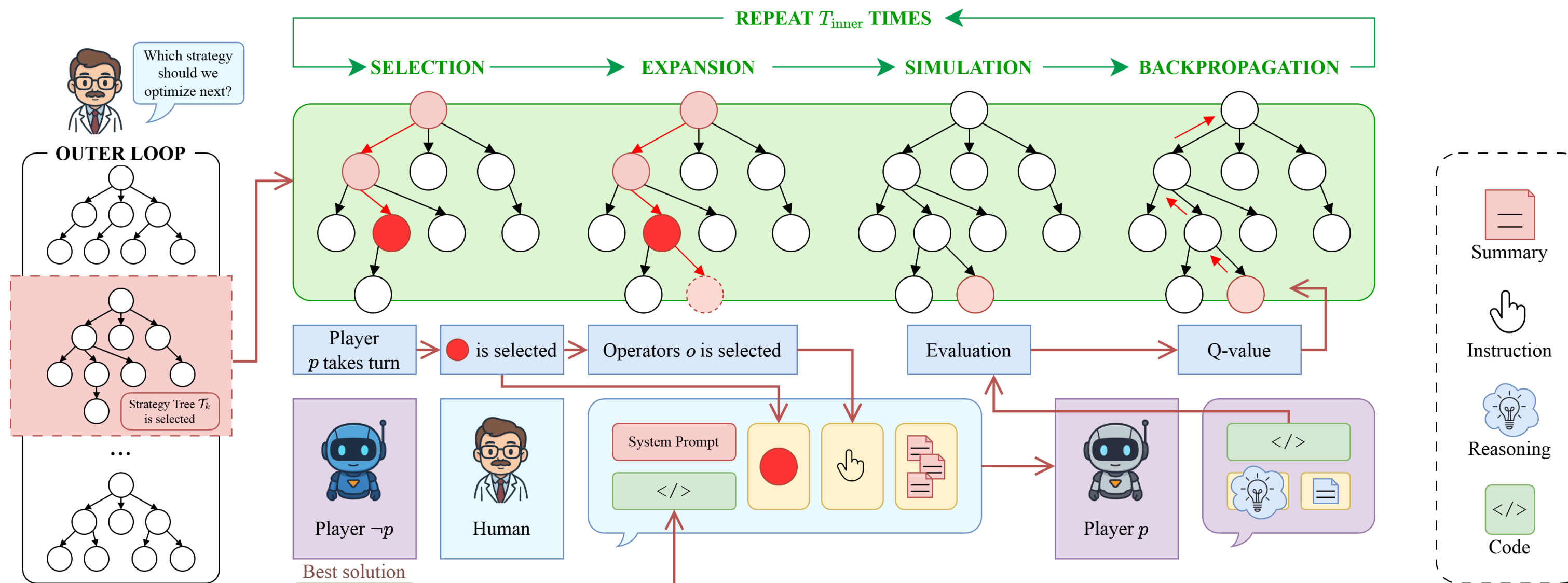


Figure 6. *Component-wise Competition.* The outer loop selects which strategy to optimize using a UCB rule, after which the two agents perform Competitive Monte Carlo Tree Search on that component. Each node represents a game state containing both agents' current implementations, and each action corresponds to applying an operator that prompts the LLM to generate a new code variant.

Why new operators?

To model *competitive*, *cooperative*, and *independently creative* behaviors between the 2 agents - capabilities that genetic operators cannot provide.

Counter

Generates a revision that exploits weaknesses in the opponent's current implementation

Learning

Integrates the opponent's successful ideas into a refined, strengthened implementation

Innovation

Introduces a new implementation that explores unexplored design directions

Figure 7. A brief description of the three prompting operators, outlining the code-generation principles for the active LLM.

EXPERIMENTS

Table 1. Algorithmic baselines and problem instances used in the experiments

Algorithms	Problems	Instance size	#Train size	#Test size
GLS	TSP	200	5	64
	TSP	50	5	64
	CVRP	50	5	64
	MKP	50	5	64
	OP	50	5	64
ACO	BPP	50	5	64
	TSP	100	5	64
	CVRP	50	5	64
	BPP	100	5	64
	BPP	100	5	64

Table 2. *Single-strategy search*, applied to GLS for TSP

Methods	TSP50	TSP100	TSP200	TSP500
EoH	0.0000 ± 0.0000	0.0012 ± 0.0009	0.0639 ± 0.0097	0.5796 ± 0.0146
ReEvo	0.0000 ± 0.0000	0.0335 ± 0.0449	0.2081 ± 0.1927	0.7918 ± 0.3051
HSEvo	0.0108 ± 0.0138	0.3095 ± 0.2363	1.1254 ± 0.6424	2.4593 ± 0.7314
MCTS-AHD	0.0000 ± 0.0000	0.0024 ± 0.0007	0.0652 ± 0.0111	0.5611 ± 0.0216
MOTIF	0.0000 ± 0.0000	0.0007 ± 0.0006	0.0610 ± 0.0100	0.5577 ± 0.0255

Figure 8. *Multi-strategy search*, applied to ACO across 5 different COPs

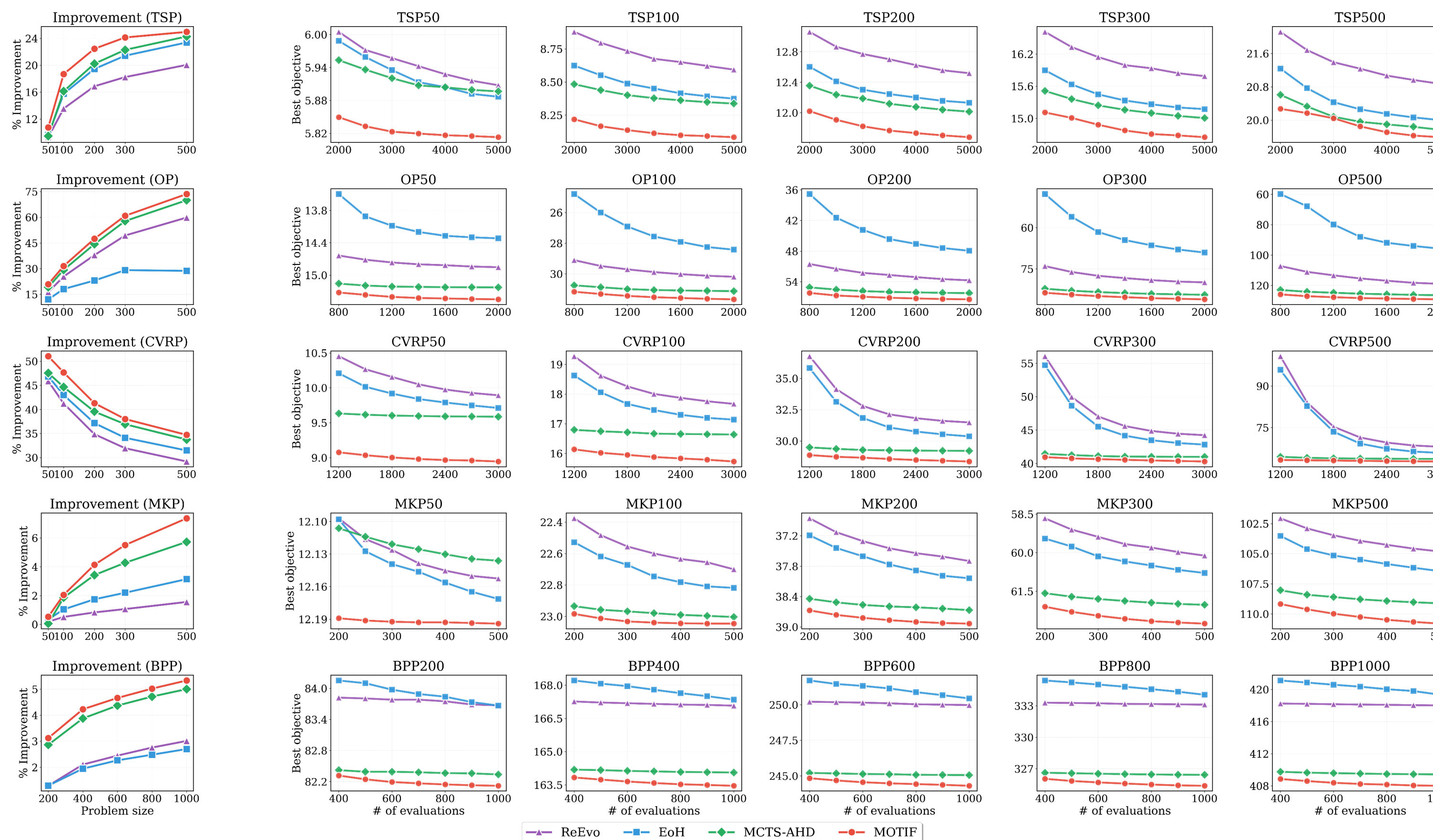
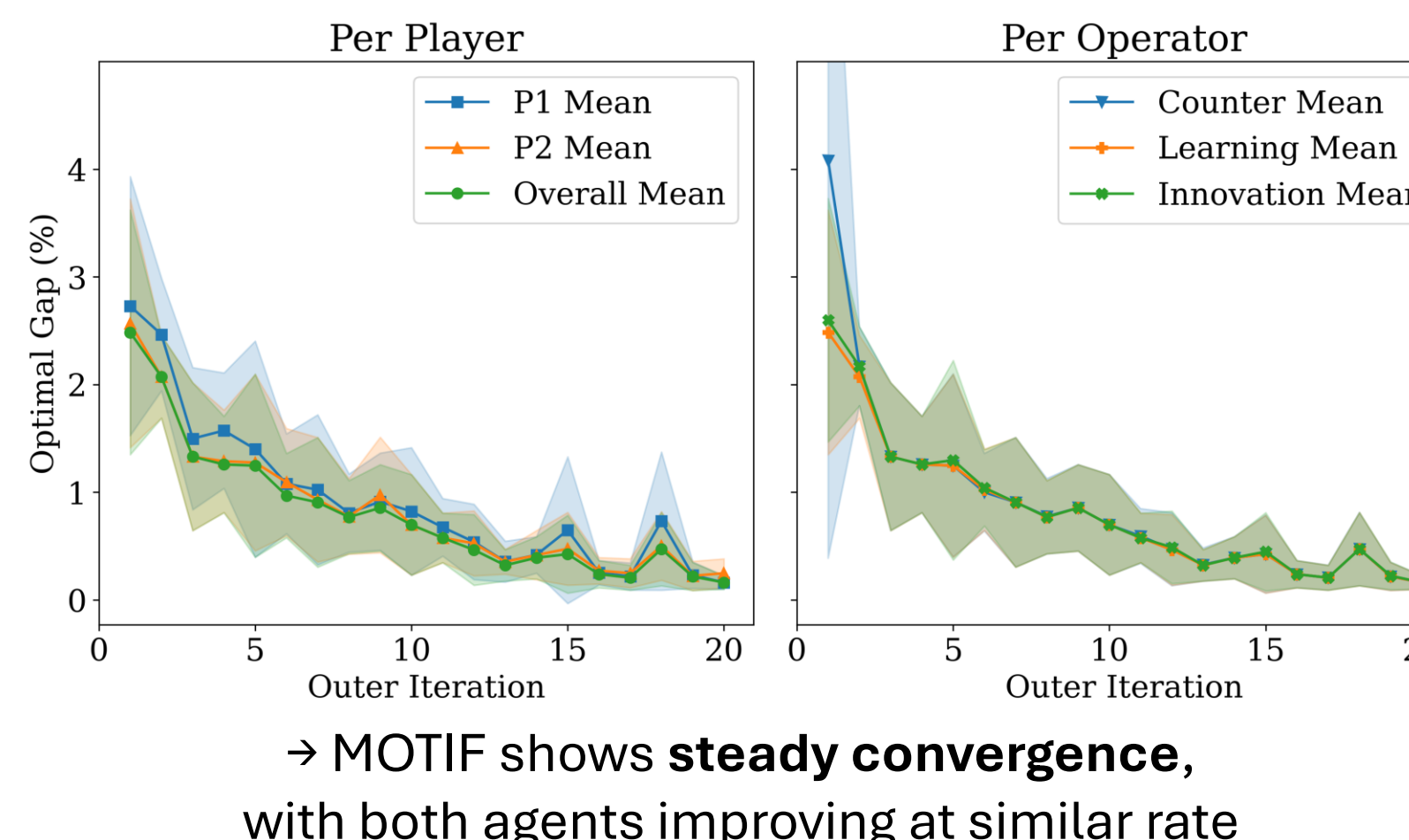


Table 3. Comparison between *single-strategy* and *multi-strategy* search in the DR algorithm on 3 COPs, showing that multi-strategy search often outperforms single-strategy search

Strategies	TSP			CVRP			BPP		
	50	100	200	50	100	200	100	200	300
π_1	0.39 ± 0.17	0.81 ± 0.27	0.72 ± 0.45	1.47 ± 0.16	2.96 ± 0.25	2.30 ± 0.39	3.17 ± 0.00	4.82 ± 0.00	4.77 ± 0.00
π_2	3.13 ± 0.05	6.24 ± 0.21	8.18 ± 0.26	4.27 ± 0.24	7.04 ± 0.73	6.85 ± 0.75	23.19 ± 0.05	24.42 ± 0.02	25.00 ± 0.00
π_3	3.88 ± 0.04	7.91 ± 0.01	11.35 ± 0.03	6.34 ± 0.08	5.07 ± 0.02	4.28 ± 0.03	12.70 ± 7.51	12.84 ± 7.15	12.15 ± 7.49
(π_1, π_2)	2.58 ± 0.50	5.04 ± 0.78	5.84 ± 1.26	6.06 ± 1.64	7.94 ± 2.71	8.85 ± 2.64	23.15 ± 0.15	24.40 ± 0.06	24.98 ± 0.02
(π_2, π_3)	3.83 ± 0.16	7.97 ± 0.14	11.59 ± 0.07	10.64 ± 0.74	12.31 ± 0.57	11.71 ± 1.03	23.05 ± 0.11	24.32 ± 0.15	24.89 ± 0.15
(π_1, π_2, π_3)	3.88 ± 0.04	7.96 ± 0.03	11.65 ± 0.05	10.98 ± 1.64	12.84 ± 3.17	13.06 ± 3.67	23.94 ± 0.55	25.02 ± 0.41	25.41 ± 0.26

Covergence and Diversity Analysis



Operator	Success rate (↑)	Novelty score (↑)	Silhouette score (↑)
Counter	93 ± 2 %	0.0136 ± 0.0067	0.5121 ± 0.0208
Learning	92 ± 2 %	0.0118 ± 0.0054	0.5325 ± 0.0300
Innovation	97 ± 1 %	0.0175 ± 0.0110	0.4793 ± 0.0240

- All 3 operators contribute effectively to progress, each with **distinct behavioral roles**
- **Learning**: yields stable and cohesive code
- **Innovation**: produces diverse, creative code
- **Counter**: balances novelty and cohesion

REFERENCES

- [1] Fei et al. "Evolution of heuristics: Towards efficient automatic algorithm design using large language model." (ICML 2024)
- [2] Ye et al. "ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution." (NeurIPS 2024)
- [3] Dat et al. "HSEvo: Elevating Automatic Heuristic Design with Diversity-Driven Harmony Search and Genetic Algorithm Using LLMs." (AAAI 2025)
- [4] Zheng et al. "Monte Carlo Tree Search for Comprehensive Exploration in LLM-Based Automatic Heuristic Design." (ICML 2025)